Varendra University

Dept. of CSE

Course Name: Parallel Processing and Distributed System Lab

Course Code: CSE 434

Lab Assignment (Spring 2026)

Total Marks: 10

Implement all the given tasks [CO3, PO6, K5]

Problem: A university wants to develop a distributed attendance management system where multiple faculty members can update and view student attendance remotely. The system will use Java RMI for remote communication and a shared server-side data structure to store attendance records. The server must handle multiple client requests simultaneously using multithreading concepts.

- The server will store all attendance records in an in-memory data structure such as a HashMap.
- Each student record will be identified using a unique Student ID.
- Attendance will be updated and retrieved through remote method calls.
- The same server data will be shared among all connected clients.

Task 1: RMI Server Implementation

Develop a Java RMI server application that provides the following remote services:

a) Mark attendance of a student (Present/Absent) using Student ID.

b) Retrieve attendance history/status of a specific student.

c) Calculate and return attendance percentage of a student.

d) Return a list of all stored student attendance records.

e) Store all attendance data in a shared server-side data structure and ensure thread-safe access for multiple clients.

Task 2: GUI-based RMI Client Application

Develop a Java GUI client application (Swing/JavaFX) that allows users to interact with the server.

a) Allow user to input:

- Student ID
- Attendance status (Present/Absent)
- Option to view attendance report [1 Mark]

b) Provide GUI controls such as:

- Text fields
- Buttons

• Selection options (radio buttons or dropdown)
• Output display area
c) Send requests to the RMI server using remote method calls.
d) Display server responses such as:
• Attendance status
• Percentage
• Full attendance record list
e) Demonstrate proper working of multiple clients accessing the same server simultaneously.

Instructions
• Use Java RMI architecture properly (Remote Interface, Implementation, Registry).
• Use a shared in-memory data structure in the server to store attendance records.
• Ensure thread-safe handling of data for concurrent client requests.
• GUI must be user-friendly and functional.
• Clean code structure, modular design, and proper comments are required.

Submission guideline:
You have to submit your source code along with any necessary configuration files.
➢ Create a folder containing the source code along with any necessary configuration files.
➢ Make a single zip file.
➢ Name the zip file with your ID number.
➢ Submit the zip file.
➢ Finally, you have to submit a **lab-report (Hard-copy)** on your project (Introduction: describing how you implemented RMI and multithreading, code, screenshot of your interface and output).